Questions-réponses d'entraînement — soutenance CIN · Avatar-ID

Comment s'entraîner 1. Lire la question. 2. Répondre **à voix haute** en 20 à 40 secondes. 3. Comparer avec la réponse proposée. 4. Marquer les questions ratées et y revenir le lendemain.

Les questions marquées ★ sont les plus probables.

A. Contexte et périmètre

1. ★ Quel était exactement votre périmètre ? > La capture de la carte : détection des contours, contrôle de qualité, reconnaissance recto/verso, autocapture, recadrage et amélioration. La vivacité, la comparaison des photos, l'OCR et l'enregistrement sont les étapes suivantes d'Avatar-ID. Je ne les ai pas développées.

2. Qu'est-ce qu'Avatar-ID ? > Une application Flutter de vérification d'identité à distance, dans l'esprit du KYC. Elle enchaîne la capture de la carte (mon module), la vivacité, la comparaison des photos, l'OCR et l'enregistrement.

3. Que transmet votre module à Avatar-ID ? > Le recto et le verso redressés et orientés, sous forme d'octets d'image (Uint8List), et la page A4 composée si besoin.

4. Pourquoi ne pas utiliser une plateforme KYC existante (Onfido, Jumio…) ? > Les données d'identité partiraient chez un tiers, avec un coût de licence, une dépendance au réseau au moment de la capture et peu de personnalisation pour la CIN tunisienne. Avatar-ID voulait un module interne, exécuté sur l'appareil.

5. Pourquoi la qualité de capture est-elle si importante ? > Parce que les étapes suivantes en dépendent : une image floue ou de travers dégrade la comparaison des visages et la lecture OCR.

B. Choix technologique

6. ★ Pourquoi OpenCV plutôt que ML Kit ? > Le Document Scanner de ML Kit n'existe que sur Android. Il aurait fallu une deuxième chaîne de détection pour iOS, alors que le cahier des charges exigeait un seul code pour les deux plateformes.

7. ★ Pourquoi pas du deep learning (un modèle entraîné) ? > Il faut plusieurs centaines d'images de CIN annotées. Pour une pièce d'identité sensible, il était impossible d'en collecter et d'en annoter autant en six semaines, et il n'existe pas de jeu public. OpenCV ne demande aucun entraînement, seulement un réglage de seuils.

8. Votre projet utilise-t-il quand même un modèle appris ? > Oui, un seul : la cascade de Haar de détection de visage. Elle est pré-entraînée et fournie par OpenCV ; je ne l'ai pas entraînée.

9. Quel est le prix à payer avec OpenCV ? > Un réglage empirique des seuils (Canny, score du quadrilatère, régions d'intérêt), obtenu par essais sur des cartes réelles plutôt qu'appris.

10. Avez-vous testé sur iOS ? > Oui : même pipeline, même comportement que sur Android. C'est justement ce qui valide le choix d'OpenCV.

11. Qu'est-ce qu'opencv_dart ? > Un binding qui permet d'appeler OpenCV (C++) depuis Dart, via FFI. Les matrices vivent en mémoire native : il faut les libérer à la main avec dispose().

C. Architecture

12. ★ Pourquoi la Clean Architecture ? > Pour séparer la logique métier (domaine, en Dart pur) des dépendances techniques (OpenCV, caméra). La machine à états et la géométrie se testent sans caméra ni émulateur, et un module ne peut pas casser silencieusement un autre.

13. Que veut dire feature-first ? > Le code est organisé par fonctionnalité (détection, qualité, autocapture…) plutôt que par type de fichier. Chaque module ne prend que les couches dont il a besoin.

14. ★ Pourquoi un isolate ? > Le traitement OpenCV est coûteux. Sur le fil principal, il faisait saccader l'aperçu caméra. L'isolate est un fil séparé, persistant, chargé une seule fois avec la cascade et les gabarits.

15. Que se passe-t-il si une image arrive pendant un traitement ? > Elle est ignorée : il n'y a pas de file d'attente. Le débit s'adapte seul à la vitesse réelle, sans accumuler de retard.

D. Détection de la carte

16. ★ Expliquez le pipeline de détection. > 1. Conversion de l'image en matrice OpenCV. > 2. CLAHE pour le contraste local. > 3. Quatre passes de masques de contours. > 4. Simplification des contours en polygones. > 5. Score géométrique pour retenir le meilleur quadrilatère (≥ 0,30). > 6. Homographie qui redresse la carte en 856 × 540.

17. Quelles sont les quatre passes ? > Canny, seuillage adaptatif, saturation HSV avec Otsu, et gradient morphologique en dernier recours.

18. Qu'est-ce que la sortie anticipée ? > Dès qu'un candidat atteint un score de 0,65, on saute les passes suivantes, plus coûteuses.

19. ★ Comment fonctionne Canny ? > 1. Flou. > 2. Gradient avec Sobel. > 3. Amincissement des bords (suppression des non-maxima). > 4. Hystérésis à deux seuils : au-dessus du seuil haut, c'est un bord ; entre les deux, il n'est gardé que s'il touche un bord fort.

20. Pourquoi des seuils à 0,66 et 1,33 fois la médiane ? > C'est une heuristique classique : les seuils s'adaptent automatiquement à la luminosité de la scène, au lieu d'être fixés à la main.

21. ★ Expliquez la formule du score. > score = 0,8 × proportion + 0,2 × taille. La proportion compare le rapport largeur/hauteur à 1,586, le format ID-1 ; le terme en 1/r la rend insensible à une carte tournée de 90°. La taille vise 30 % de l'image.

22. Que fait approxPolyDP ? > Il simplifie un contour en polygone (algorithme de Douglas-Peucker), avec une tolérance de 2 % du périmètre. Quatre sommets convexes donnent un candidat net ; sinon, on prend le rectangle englobant minimal, avec une pénalité de 0,85.

23. Quels garde-fous contre une fausse détection ? > On rejette : > - les quadrilatères collés au bord de l'image ; > - ceux qui couvrent plus de 75 % de l'image ; > - ceux qui épousent mal leur contour. > > Il faut en plus plusieurs images consécutives pour confirmer.

24. Qu'est-ce qu'une homographie ? > Une transformation de perspective, représentée par une matrice 3 × 3 et calculée à partir de 4 paires de points. Elle « remet à plat » la carte vue de biais.

25. Pourquoi une recherche suivie ? > Une fois la carte trouvée, on cherche seulement à ±20 % autour de sa position précédente : c'est plus rapide et cela évite le tremblement.

26. Pourquoi réduire l'image à 45 % dans la première passe ? > Pour la vitesse. Le facteur a été relevé de 0,30 à 0,45 parce qu'une réduction trop forte effaçait le bord faible d'une carte claire.

E. La difficulté du fond clair

27. ★ Quelle a été votre principale difficulté ? > La détection échouait sur une carte claire posée sur un fond clair. Deux causes : > - un bug : les seuils de Canny étaient calculés sur la moyenne au lieu de la médiane ; > - un contraste local trop faible. > > Corrigé par la vraie médiane, un CLAHE et un nouveau réglage des seuils.

28. Pourquoi la moyenne posait-elle problème ? > Sur une scène claire, la moyenne est tirée vers le haut. Les seuils de Canny montent trop, et un bord faible n'est plus détecté.

29. Qu'est-ce que CLAHE ? > *Contrast Limited Adaptive Histogram Equalization* : une égalisation d'histogramme faite par tuiles (localement), avec un plafond sur l'amplification pour ne pas exagérer le bruit.

30. Lequel des trois correctifs a le plus compté ? > Je ne l'ai pas isolé. La bonne méthode serait une étude d'ablation : tester chaque correctif seul sur un jeu d'images fixe.

F. Qualité et autocapture

31. ★ Comment mesurez-vous la netteté ? > Par la variance du Laplacien (dérivée seconde) sur la carte redressée, après CLAHE. Une image nette a des bords forts, donc une variance élevée. Le seuil est de 120.

32. Pourquoi 120 ? > C'est un seuil empirique, valable uniquement à la résolution de redressement 856 × 540. À une autre résolution, la variance change.

33. Comment détectez-vous un reflet ? > Par la fraction de pixels saturés (≥ 250) : au-delà de 12 %, c'est un reflet, même si la luminosité moyenne paraît correcte.

34. Comment mesurez-vous la stabilité ? > Par la dérive moyenne des 4 coins entre deux images, divisée par la diagonale du quadrilatère, ce qui la rend indépendante de la résolution. Il faut rester sous 2,5 % pendant 3 images.

35. ★ Décrivez la machine à états. > - **searching** : aucune carte ; > - **poorQuality** : un contrôle échoue ; > - **holding** : tout est bon, on maintient ; > - **triggerCapture**, puis **captured**. > > Carte perdue → retour à searching ; contrôle raté → retour à poorQuality.

36. Pourquoi un compteur de deux images plutôt qu'une minuterie ? > Pour ne déclencher que sur des conditions confirmées, et pas sur un seul instant favorable. Un compteur s'adapte aussi au débit réel.

37. Pourquoi environ 2 s si deux images suffisent ? > À environ 10 analyses par seconde, les compteurs ne font qu'une fraction de seconde. Le reste vient de la confirmation des signaux de contenu et du geste de l'utilisateur. C'est un chronométrage manuel.

G. Recto / verso

38. ★ Comment reconnaissez-vous le recto ? > - Un visage, cherché en haut à gauche avec la cascade de Haar, sur 4 rotations ; la bonne rotation donne aussi l'orientation. > - Puis le logo et le drapeau, confirmés par template matching (corrélation ≥ 0,6).

39. ★ Comment fonctionne une cascade de Haar ? > - Des caractéristiques de Haar (différences entre rectangles clairs et sombres), calculées très vite grâce à l'image intégrale. > - AdaBoost choisit les meilleures caractéristiques. > - Une cascade de classifieurs rejette vite les zones sans visage.

40. Que signifient scaleFactor 1,05 et minNeighbors 8 ? > On agrandit la fenêtre de recherche de 5 % à chaque échelle, et il faut 8 détections qui se recoupent pour valider un visage, ce qui limite les faux positifs.

41. Qu'est-ce que le template matching ? > On fait glisser une image de référence (le gabarit) sur une région et on mesure la ressemblance. TM_CCOEFF_NORMED est une corrélation normalisée, insensible aux changements globaux de luminosité.

42. ★ Comment reconnaissez-vous le verso ? > Trois indices votent sur 4 rotations : > - le code-barres, localisé en bonne position (2 points) ; > - la texture de l'empreinte, avec une variance du Laplacien dans [150 ; 4 000] (1 point) ; > - la ligne de séparation, trouvée par Hough (1 point).

43. Qu'est-ce que la transformée de Hough ? > Chaque point de bord vote pour les droites qui pourraient passer par lui ; les droites qui reçoivent beaucoup de votes existent. La version probabiliste renvoie directement des segments.

44. Pourquoi limiter Hough au bas de la carte ? > Sinon les lignes de texte, le bord de la photo ou celui du code-barres concurrencent la vraie ligne de séparation.

45. Comment évitez-vous de capturer deux fois le recto ? > Après le recto, seule une face qui réunit les trois indices du verso peut déclencher la capture.

H. Résultats et tests

46. ★ Quels sont vos résultats ? > - 92 à 100 ms par image (cible < 100) ; > - 92 % de recto/verso reconnus (cible ≥ 90 %) ; > - environ 2 s jusqu'à la capture ; > - 177 tests automatisés réussis ; > - même comportement sur Android et iOS.

47. 92–100 ms, est-ce la latence de bout en bout ? > Non : c'est le temps de traitement d'une image dans l'isolate, horodaté avant/après. L'aperçu reste fluide grâce à l'isolate.

48. Sur combien de cartes et quel téléphone ? > Préparez votre réponse exacte : modèles Android et iOS, nombre de cartes, nombre d'essais. Le rapport ne le précise pas.

49. ★ Comment sont organisés vos tests ? > En quatre paliers : A, logique pure Dart ; B, plateforme simulée avec des mocks ; C, OpenCV réel sur images synthétiques ; D, vraies cartes sur téléphone, avec un protocole manuel.

50. Quelle est votre couverture de code ? > Elle n'a pas été mesurée ; `flutter test --coverage` le permet directement. La suite compte 28 fichiers et la dernière campagne a exécuté 177 cas, tous réussis.

51. Qu'est-ce qu'une image synthétique dans un test ? > Une image générée par le test lui-même (un rectangle dessiné, par exemple), dont on connaît la bonne réponse. Elle permet de tester OpenCV sans caméra ni vraie carte.

I. Sécurité, limites et perspectives

52. ★ Une photo d'écran peut-elle passer ? > Les contrôles (capture en direct, deux faces distinctes, qualité) réduisent fortement ce risque. La vivacité de la personne est ensuite vérifiée par Avatar-ID. Une analyse de texture de la carte renforcerait encore la chaîne.

53. Où vont les images ? > Dans la galerie de l'appareil, puis dans l'historique local, avec suppression possible. Tout le traitement se fait sur l'appareil, sans service tiers.

54. Est-ce prêt pour la production ? > Le module atteint ses cibles. Pour la production, je propose : > - un jeu d'images plus large ; > - des tests réels automatisés ; > - un message de timeout ; > - une analyse anti-fraude de la carte.

55. Quelle serait votre première priorité ? > Constituer un jeu d'images de CIN plus représentatif. Il sert à la fois à affiner les seuils, à automatiser les tests réels et, à terme, à envisager un modèle appris.

56. Qu'avez-vous développé vous-même, par rapport à ce que fournit OpenCV ? > OpenCV fournit les opérateurs : Canny, Otsu, CLAHE, contours, homographie, Haar, Hough, détecteur de code-barres. J'ai conçu : > - l'enchaînement des passes et la sortie anticipée ; > - le score du quadrilatère, le suivi et les garde-fous ; > - l'isolate ; > - la stabilisation et la machine à états ; > - le vote recto/verso ; > - le post-traitement, la page A4 et les écrans.

57. Si c'était à refaire, que changeriez-vous ? > Je constituerais plus tôt un petit jeu d'images de référence, fixe, pour mesurer chaque réglage avant/après et automatiser une partie des tests réels.

58. Qu'avez-vous appris ? > La vision par ordinateur appliquée, Flutter multiplateforme, la Clean Architecture et les tests structurés. Surtout, qu'un choix technique se justifie par les contraintes réelles d'un projet.